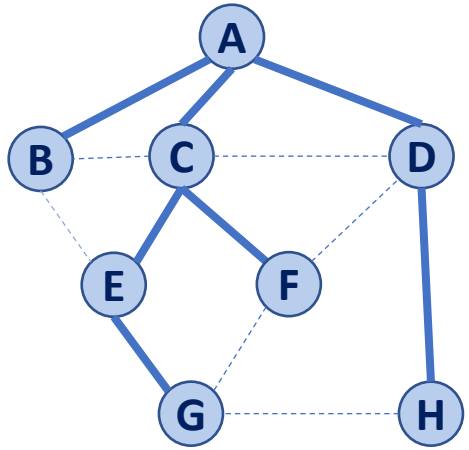
CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Traversal: BFS



d	p		Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G

——— Discovery

• • • • • Cross

BFS Analysis

Q1: Does our implementation handle disjoint graphs? If so, what code handles this? yes :10 -12

- ***How do we use this to count components?***

Line 12 compCt++;

Q2: Does our implementation detect a cycle? yes

- ***How do we use this to detect a cycle?***

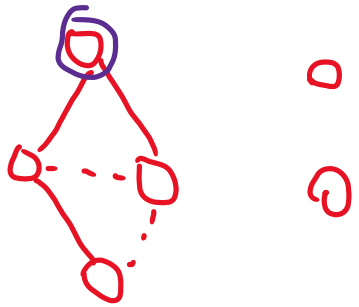
Line 27 cycle = true;

Q3: What is the running time? O(n+m) Ideal

```
1   BFS(G):
2     Input: Graph, G
3     Output: A labeling of the edges on
4         G as discovery and cross edges
5
6     foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8     foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10    foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12        BFS(G, v)
```

```
14  BFS(G, v):
15    Queue q
16    setLabel(v, VISITED)
17    q.enqueue(v)
18
19    while !q.empty():
20      v = q.dequeue()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23          setLabel(v, w, DISCOVERY)
24          setLabel(w, VISITED)
25          q.enqueue(w)
26        elseif getLabel(v, w) == UNEXPLORED:
27          setLabel(v, w, CROSS)
```

→ Init



Ensure BFS

? runs on every comp?

compCt_++;

cycle_=true;

```
1   BFS(G):
2     Input: Graph, G
3     Output: A labeling of the edges on
4         G as discovery and cross edges
5
6     foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8     foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10    foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12        BFS(G, v)
```

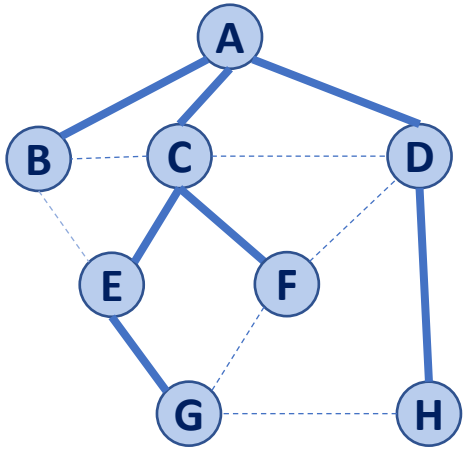
$$\sum_{v \in G} deg(v) = 2m$$

```
14  BFS(G, v):
15    Queue q
16    setLabel(v, VISITED)
17    q.enqueue(v)
18
19    while !q.empty():
20      v = q.dequeue()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23          setLabel(v, w, DISCOVERY)
24          setLabel(w, VISITED)
25          q.enqueue(w)
26        elseif getLabel(v, w) == UNEXPLORED:
27          setLabel(v, w, CROSS)
```

n elem.

deg(v)

Running time of BFS



Incomplete

$O(n^2)$

While-loop at **:19**?

n total runs

For-loop at **:21**?

2m total runs

$n + 2m = O(n + m)$

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G

BFS Observations

Q1: What is a shortest path from **A** to **H**? d = 2

Path found via D from H

Q2: What is a shortest path from **E** to **H**? We don't know

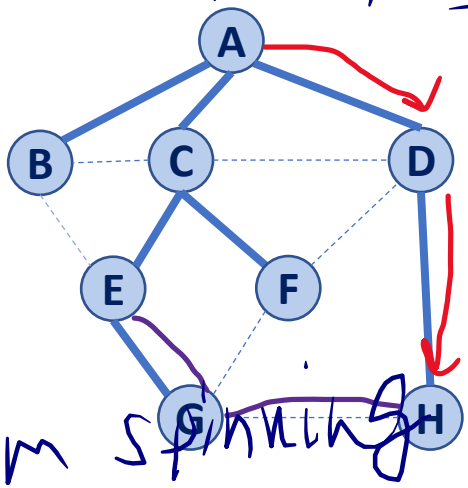
→ BFS gives us a single source shortest path SSSP

Q3: How does a cross edge relate to **d**?

$|d(u) - d(v)| \leq 1$

Q4: What structure is made from discovery edges? Minimum spanning tree (MST)

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

Obs. 1: Traversals can be used to count components.

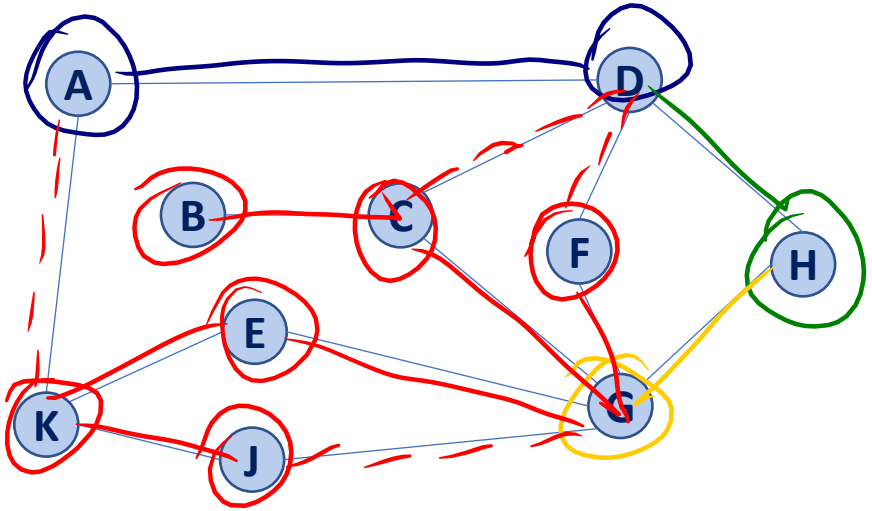
Obs. 2: Traversals can be used to detect cycles.

Obs. 3: In BFS, **d** provides the shortest distance to every vertex.

Obs. 4: In BFS, the endpoints of a cross edge never differ in distance, **d**, by more than 1:

$$|d(u) - d(v)| \leq 1$$

Traversal: DFS



Discovery

Back

```
1   BFS(G):
2     Input: Graph, G
3     Output: A labeling of the edges on
4         G as discovery and cross edges
5
6     foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8     foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10    foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12        BFS(G, v)
```

```
14  BFS(G, v):
15    Queue q
16    setLabel(v, VISITED)
17    q.enqueue(v)
18
19    while !q.empty():
20      v = q.dequeue()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23          setLabel(v, w, DISCOVERY)
24          setLabel(w, VISITED)
25          q.enqueue(w)
26        elseif getLabel(v, w) == UNEXPLORED:
27          setLabel(v, w, CROSS)
```

DFS

DFS

DFS

DFS(G,w)

BACK

```
1  DFS(G):
2    Input: Graph, G
3    Output: A labeling of the edges on
4        G as discovery and back edges
5
6    foreach (Vertex v : G.vertices()):
7      setLabel(v, UNEXPLORED)
8    foreach (Edge e : G.edges()):
9      setLabel(e, UNEXPLORED)
10   foreach (Vertex v : G.vertices()):
11     if getLabel(v) == UNEXPLORED:
12       DFS(G, v)
```

```
14 DFS(G, v):
15   Queue q
16   setLabel(v, VISITED)
17   q.enqueue(v)
18
19   while !q.empty():
20     v = q.dequeue()
21     foreach (Vertex w : G.adjacent(v)):
22       if getLabel(w) == UNEXPLORED:
23         setLabel(v, w, DISCOVERY)
24         setLabel(w, VISITED)
25         DFS(G, w)
26       elseif getLabel(v, w) == UNEXPLORED:
27         setLabel(v, w, BACK)
```

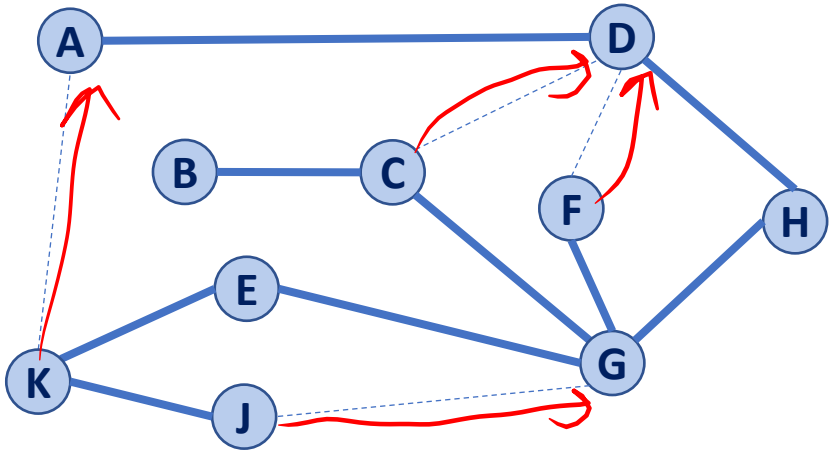
CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Traversal: DFS



① How vertices are visited

② The non-discovery edge

Discovery Edge

Back Edge

```
14  DFS(G, v):
15    Stack s
16    setLabel(v, VISITED)
17    s.push(v)
18
19    while !s.empty():
20      v = s.pop()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23           setLabel(v, w, DISCOVERY)
24           setLabel(w, VISITED)
25           s.push(w)
26        elseif getLabel(v, w) == UNEXPLORED:
27           setLabel(v, w, BACK)
```

```
14  DFS(G, v):
15    Queue q
16    setLabel(v, VISITED)
17    q.enqueue(v)
18
19    while !q.empty():
20      v = q.dequeue()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23           setLabel(v, w, DISCOVERY)
24           setLabel(w, VISITED)
25           DFS(G, w)
26        elseif getLabel(v, w) == UNEXPLORED:
27           setLabel(v, w, BACK)
```

```
1   BFS(G):
2     Input: Graph, G
3     Output: A labeling of the edges on
4         G as discovery and cross edges
5
6     foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8     foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10    foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12        BFS(G, v)
```

O(n)

O(m)

```
14  BFS(G, v):
15    Queue q
16    setLabel(v, VISITED)
17    q.enqueue(v)
18
19    while !q.empty():
20      v = q.dequeue()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23          setLabel(v, w, DISCOVERY)
24          setLabel(w, VISITED)
25          q.enqueue(w)
26        elseif getLabel(v, w) == UNEXPLORED:
27          setLabel(v, w, CROSS)
```

Running time of DFS

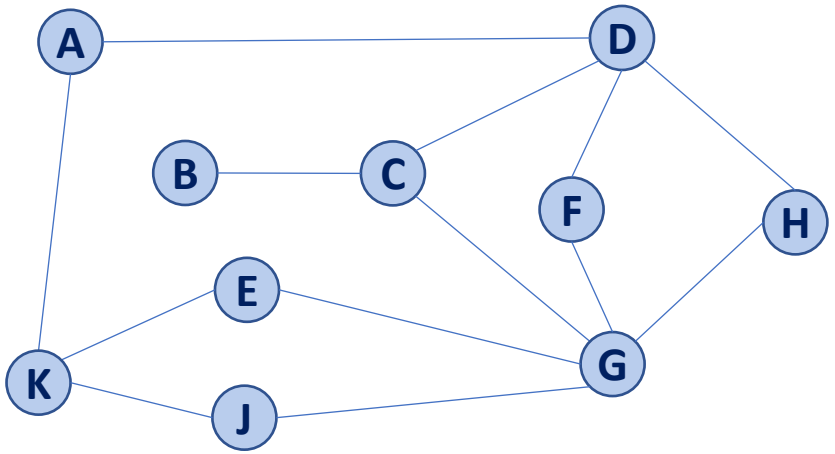
Labeling:

- Vertex: $O(n)$
- Edge: $O(m)$

Queries:

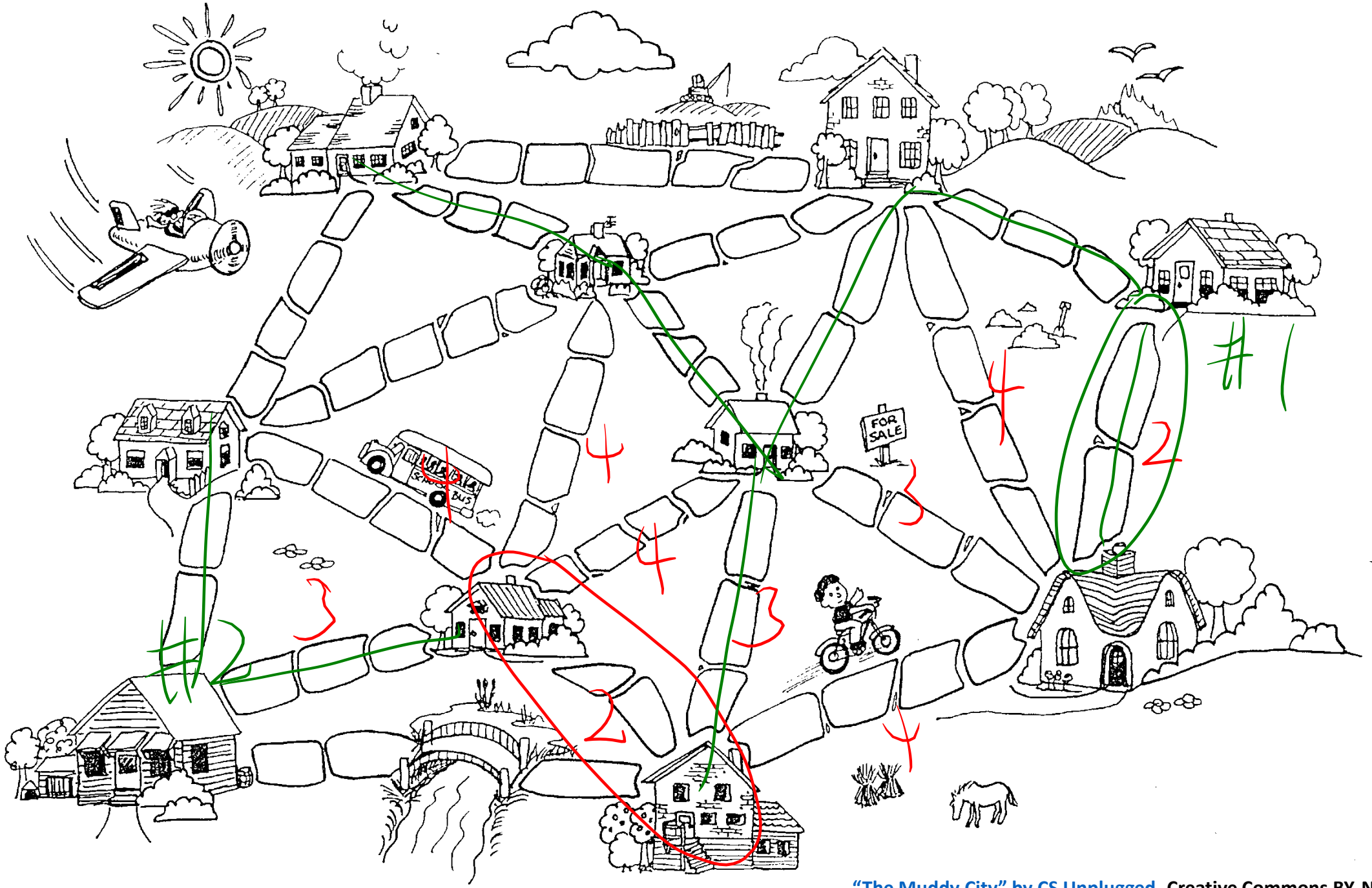
- Vertex: $O(n)$
- Edge: $O(m)$

total $O(n+m)$




```
14  DFS(G, v):
15    setLabel(v, VISITED)
16    foreach (Vertex w : G.adjacent(v)):
17      if getLabel(w) == UNEXPLORED:
18        setLabel(v, w, DISCOVERY)
19        DFS(G, w)
20      elseif getLabel(v, w) == UNEXPLORED:
21        setLabel(v, w, BACK)
```

$\deg(v)$

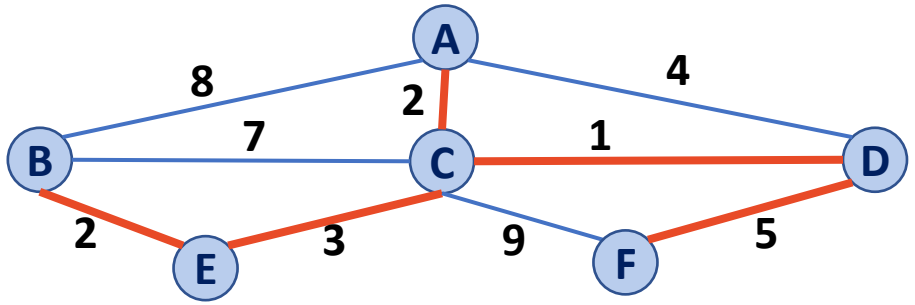


Minimum Spanning Tree Algorithms

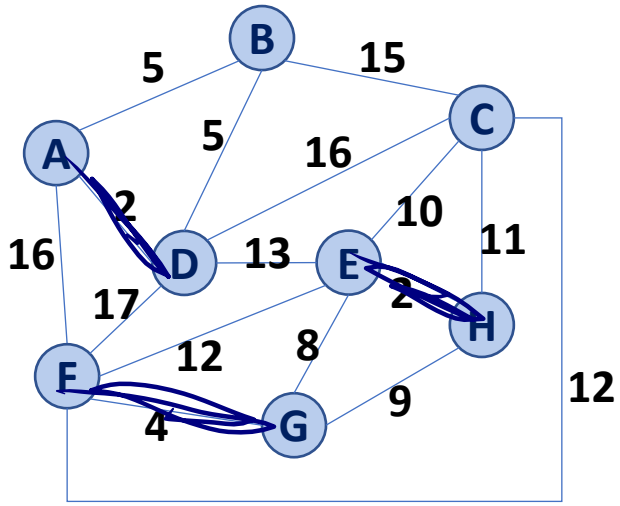
Input: Connected, undirected graph **G** with edge weights (unconstrained, but must be additive)

Output: A graph G' with the following properties:

- G' is a spanning graph of G
- G' is a tree (connected, acyclic)
- G' has a minimal total weight among all spanning trees



Kruskal's Algorithm



PQ

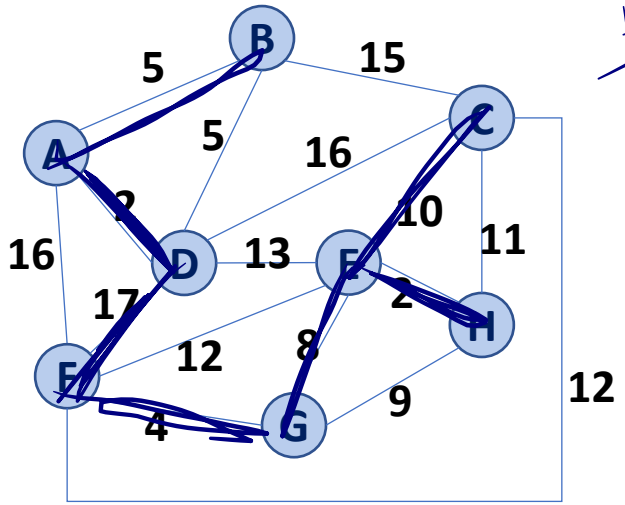
(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

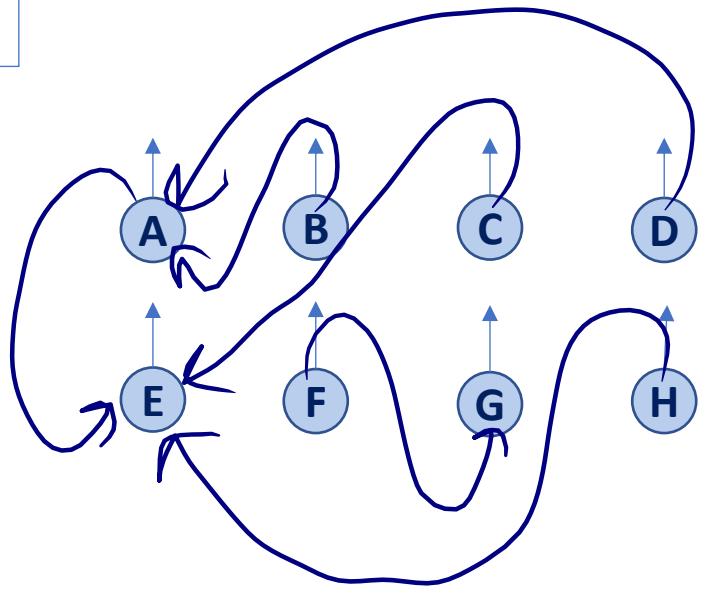
weighted order

- Heap
- Sorted array

Kruskal's Algorithm

Stop when # of edges in G' == n-1





DS

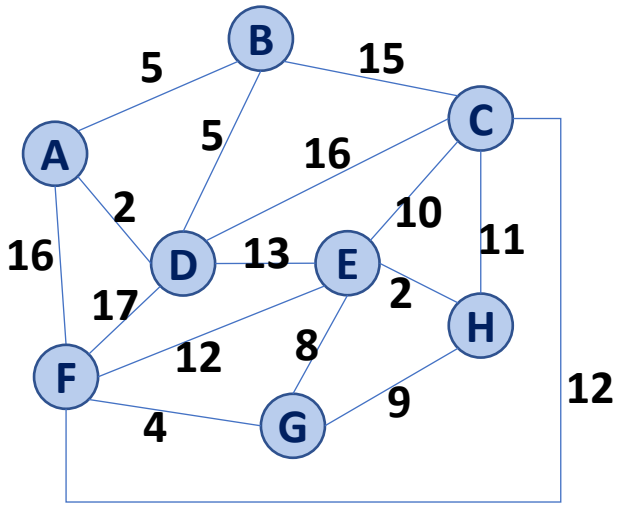
~~(A, D)~~
~~(E, H)~~
~~(F, G)~~
~~(A, B)~~
(B, D)
~~(G, E)~~
(G, H)
~~(E, C)~~
(C, H)
(E, F)
(F, C)
~~(D, E)~~
~~(B, C)~~
(C, D)
(A, F)
(D, F)

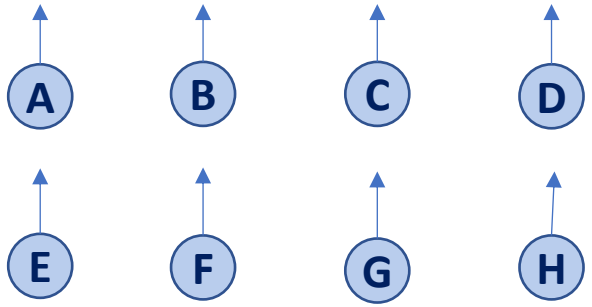
skip
skip
skip
skip

Done!

Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)






```
1  KruskalMST(G):
2    DisjointSets forest
3    foreach (Vertex v : G):
4      forest.makeSet(v)
5
6    PriorityQueue Q    // min edge weight
7    foreach (Edge e : G):
8      Q.insert(e)
9
10   Graph T = (V, {})
11
12   while |T.edges()| < n-1:
13     Edge (u, v) = Q.removeMin()
14     if forest.find(u) != forest.find(v):
15       T.addEdge(u, v)
16       forest.union( forest.find(u),
17                     forest.find(v) )
18
19   return T
```

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(m)$	$O(m \lg(n))$
Each removeMin :13	$O(\lg(n))$	$O(1)$

$m = O(n^2) \leq c * n^2$

$\lg(m) \leq \lg(c * n^2)$

$\lg(m) \leq \lg c + 2\lg(n)$

$\lg(m) = O(\lg(n))$

```
 1  KruskalMST(G):
 2    DisjointSets forest
 3    foreach (Vertex v : G):
 4      forest.makeSet(v)
 5
 6    PriorityQueue Q     // min edge weight
 7    foreach (Edge e : G):
 8      Q.insert(e)
 9
10    Graph T = (V, {})
11
12    while |T.edges()| < n-1:
13      Edge (u, v) = Q.removeMin()
14      if forest.find(u) != forest.find(v):
15         T.addEdge(u, v)
16         forest.union( forest.find(u),
17                       forest.find(v) )
18
19    return T
```

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	$O(n+m) + O(m \log n)$
Sorted Array	$O(n+m \log n) + O(m)$

```
1   KruskalMST(G):
2     DisjointSets forest
3     foreach (Vertex v : G):
4       forest.makeSet(v)
5
6     PriorityQueue Q     // min edge weight
7     foreach (Edge e : G):
8       Q.insert(e)
9
10    Graph T = (V, {})
11
12    while |T.edges()| < n-1:
13      Edge (u, v) = Q.removeMin()
14      if forest.find(u) != forest.find(v):
15         T.addEdge(u, v)
16         forest.union( forest.find(u),
17                       forest.find(v) )
18
19    return T
```

O(n) O(n) O(m)

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Kruskal's Algorithm

Priority Queue:	Total Running Time
Heap	O(n + m) + O(m lg(n))
Sorted Array	O(n + m lg(n)) + O(m)

Heap – allows changing edge weights in O(lg(n))

Sorted array – General

```
1   KruskalMST(G):
2     DisjointSets forest
3     foreach (Vertex v : G):
4       forest.makeSet(v)
5
6     PriorityQueue Q     // min edge weight
7     foreach (Edge e : G):
8       Q.insert(e)
9
10    Graph T = (V, {})
11
12    while |T.edges()| < n-1:
13      Edge (u, v) = Q.removeMin()
14      if forest.find(u) != forest.find(v):
15         T.addEdge(u, v)
16         forest.union( forest.find(u),
17                       forest.find(v) )
18
19    return T
```

Kruskal's Algorithm

Which Priority Queue Implementation is better for running Kruskal's Algorithm?

- Heap: Flexible (change edge weights)

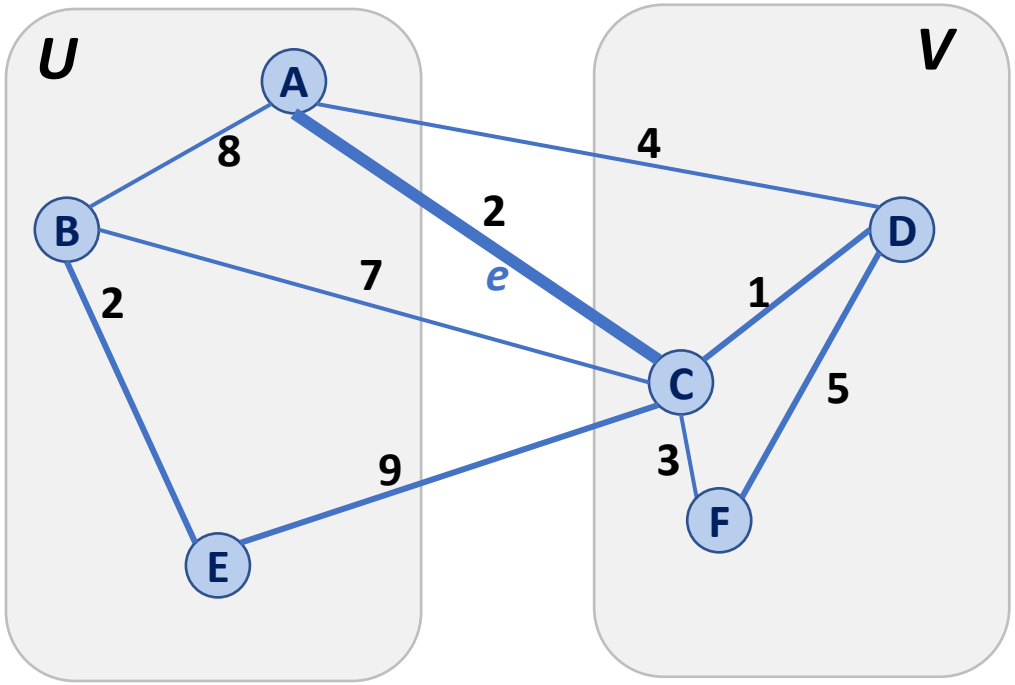
- Sorted Array: General

Partition Property

Consider an arbitrary partition of the vertices on **G** into two subsets **U** and **V**.

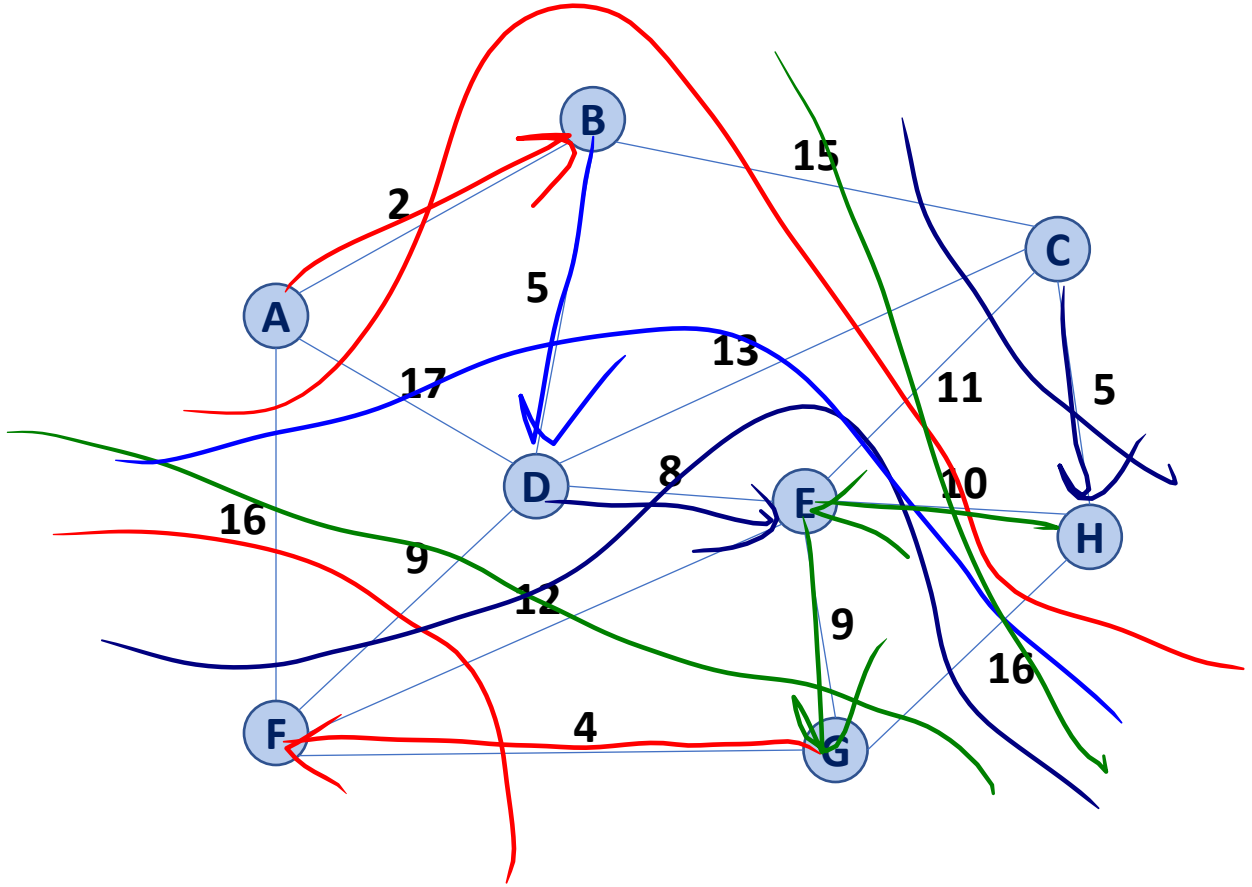
Let **e** be an edge of minimum weight across the partition.

Then **e** is part of some minimum spanning tree.

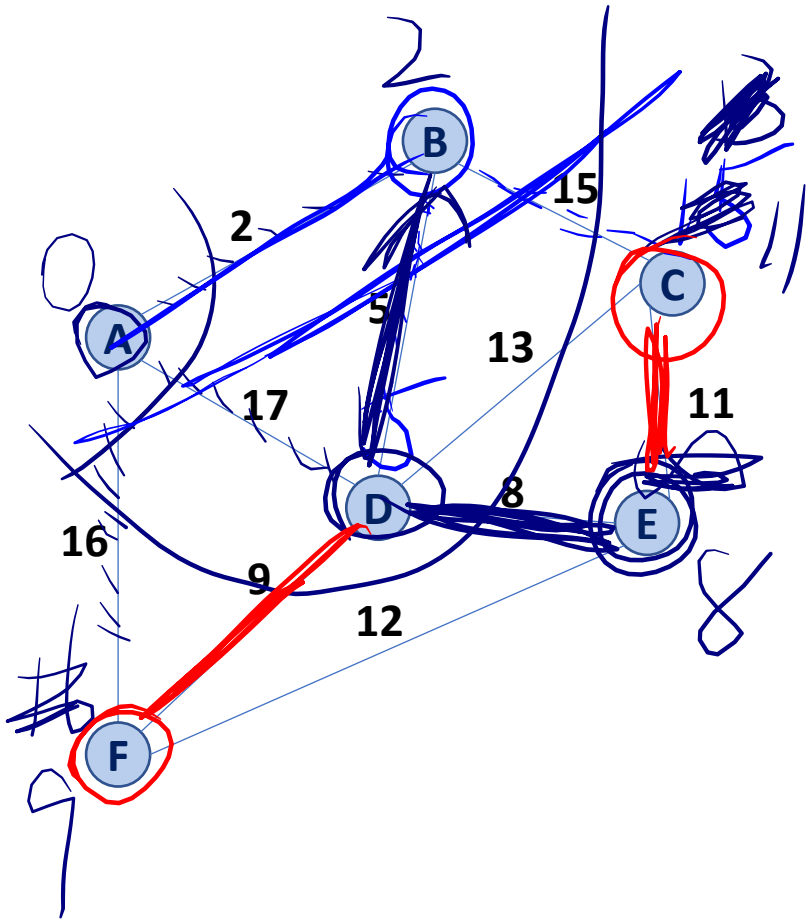


Partition Property

The partition property suggests an algorithm:



Prim's Algorithm




```
 1  PrimMST(G, s):
 2    Input: G, Graph;
 3           s, vertex in G, starting vertex
 4    Output: T, a minimum spanning tree (MST) of G
 5
 6    foreach (Vertex v : G):
 7      d[v] = +inf
 8      p[v] = NULL
 9    d[s] = 0
10
11    PriorityQueue Q   // min distance, defined by d[v]
12    Q.buildHeap(G.vertices())
13    Graph T           // "labeled set"
14
15    repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19        if cost(v, m) < d[v]:
20          d[v] = cost(v, m)
21          p[v] = m
22
23    return T
```

Prim's Algorithm

```
 6  PrimMST(G, s):
 7    foreach (Vertex v : G):
 8      d[v] = +inf
 9      p[v] = NULL
10    d[s] = 0
11
12    PriorityQueue Q // min distance, defined by d[v]
13    Q.buildHeap(G.vertices())
14    Graph T           // "labeled set"
15
16    repeat n times:
17      Vertex m = Q.removeMin()
18      T.add(m)
19      foreach (Vertex v : neighbors of m not in T):
20        if cost(v, m) < d[v]:
21          d[v] = cost(v, m)
22          p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + m\lg(n))$	$O(n\lg(n) + m\lg(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

Prim's Algorithm

Sparse Graph:

$n \sim m$

Dense Graph:

$n^2 \sim m$

```
 6  PrimMST(G, s):
 7    foreach (Vertex v : G):
 8      d[v] = +inf
 9      p[v] = NULL
10    d[s] = 0
11
12    PriorityQueue Q // min distance, defined by d[v]
13    Q.buildHeap(G.vertices())
14    Graph T          // "labeled set"
15
16    repeat n times:
17      Vertex m = Q.removeMin()
18      T.add(m)
19      foreach (Vertex v : neighbors of m not in T):
20        if cost(v, m) < d[v]:
21          d[v] = cost(v, m)
22          p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + m \log(n))$	$O(n \log(n) + m \log(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

Prim's Algorithm

Sparse Graph:

$n \sim m$

Dense Graph:

$n^2 \sim m$

```
 6  PrimMST(G, s):
 7    foreach (Vertex v : G):
 8      d[v] = +inf
 9      p[v] = NULL
10    d[s] = 0
11
12    PriorityQueue Q // min distance, defined by d[v]
13    Q.buildHeap(G.vertices())
14    Graph T          // "labeled set"
15
16    repeat n times:
17      Vertex m = Q.removeMin()
18      T.add(m)
19      foreach (Vertex v : neighbors of m not in T):
20        if cost(v, m) < d[v]:
21          d[v] = cost(v, m)
22          p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + n \log(n))$	~~$O(n \log(n) + m \log(n))$~~ $O(n(\log(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

MST Algorithm Runtime:

- Kruskal's Algorithm:

  O(n + m log(n))

- Prim's Algorithm:

  O(n log(n) + m log(n))

- What must be true about the connectivity of a graph when running an MST algorithm?

  Min edges $n-1$

  max edges n^2

  $O(n) \sim o(m)$ $m \geq n-1$

  - How does n and m relate?

MST Algorithm Runtime:

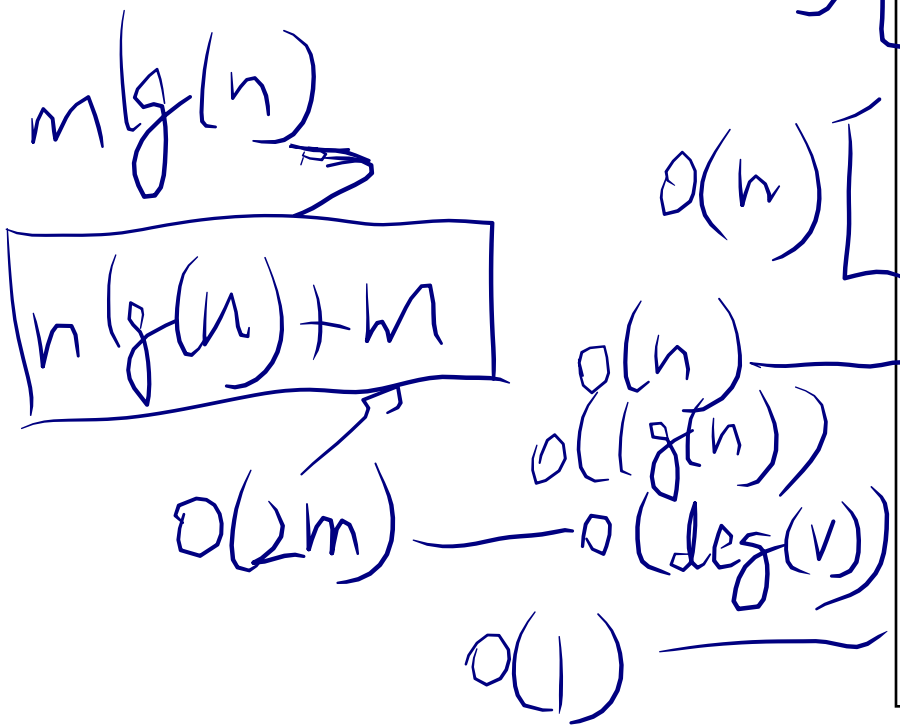
- Upper bound on MST Algorithm Runtime:

  O(m log(n))

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	O(log(n))	O(log(n))
Decrease Key	O(log(n))	O(1)*

What's the updated running time?




```
PrimMST(G, s):
  foreach (Vertex v : G):
    d[v] = +inf
    p[v] = NULL
  d[s] = 0

  PriorityQueue Q // min distance, defined by d[v]
  Q.buildHeap(G.vertices())
  Graph T          // "labeled set"

  repeat n times:
    Vertex m = Q.removeMin()
    T.add(m)
    foreach (Vertex v : neighbors of m not in T):
      if cost(v, m) < d[v]:
        d[v] = cost(v, m)
        p[v] = m
```